Tien Dao V20748106 Dat Tran V11708956

N-grams Theory:

Question 3:

a/ A bigram model using Markov assumption:

$$P(w_k | w_1, \ldots, w_{k-1}) \approx P(w_k | w_{k-1}) \quad (k \geq 2)$$

Apply it to the full joint probability:

$$P(w_1, \ldots, w_n) = \prod_{k=2}^{n} P(w_k | w_{k-1})$$

b/ Mathematically, moving to a bigram model means the word is only conditionally dependent on the word before it, and all the other previous words are ignored. Linguistically speaking, this makes it hard to model long-distance dependencies, for example long-distance grammar, meaning/topic context, or subject-verb agreement.

Example: The bouquet of roses on the table is beautiful. A bigram model might capture "table" or "roses" instead of "bouquet".

c/ A bigram model can perform reasonably well in scenarios such as:

1| Short queries / commands: "best restaurant near me", "turn on lights", etc.

2| Repetitive / template-like texts: customer service scripts, weather reports, etc. where lots of phrases repeats and is often strongly suggested by the prev. words.

Question 4:

a| Perplexity is computed from the avg. negative log probability that your model assigns to each test tokens:

$$\text{Perplexity} = \exp\left(-\frac{1}{N}\sum_{i=1}^{N}\log p(w_i \mid w_{1:i-1})\right)$$

If the test tokens are also in the training data, the model will assign them higher probabilities, which would result in lower log lost, making the perplexity smaller. This is "cheating" as you are evaluating the model on already seen data instead of new unseen data.

b/ This practice is scientifically invalid as you are using the test set to make evaluation decisions. That would leak information of the test set to the model, making its perplexity higher and more biased towards the test data. Additionally, testing should be a one-time final evaluation, not repeatedly done to modify smoothing parameters as that is "cheating".

c/ A development (dev) set is a "practice" set you use while building the model. You first train on the training set, then evaluate on the dev set in order to choose things such as smoothing methods, modify hyperparameters and n-gram orders (bigram vs trigrams), etc.

Question 5:

a/ Perplexity is how surprised the model is for the next word. The lower the perplexity the better the model is at predicting the next token. Looking at the table:

- Unigram model ($P(w_1 w_2 \ldots w_n) \approx \prod_i P(w_i)$) is modelled only on the word frequencies in the training set and ignore surrounding context completely. Therefore it would not perform well when it sees new examples → higher perplexity as it assign higher log probabilities (or 0 for unseen words) for new examples.

- Bigram models looks at the previous word. So it can model common pairs like "New York", "according to". Therefore the perplexity is much lower compared to unigrams (170 compared to 962).

- Trigams look at two words before it, allowing it to model words better ("as a result", "in the middle", etc), allowing it to have lower perplexity than bigrams and making better predictions.

b/ A 5-gram can perform worse than a trigram model if trained on a small corpus because for 5-grams to work better, it needs much more training data in order to estimate probabilities reliably, as it needs to model many 4-word histories. when it sees unseen test data, it would assign higher log probabilities to sequences that might not fit the 5-word form, or worse assign 0 probabilities on unknown tokens, making perplexity undefined. A trigam will perform more reliably trained on the small corpus as it needs less training data to capture word structure and context V. so perplexity is lower so a trigam will predict better on small data.

1)

a)

- Apply Bayes' rule, we have: $P(c|d) = \frac{P(d|c)\,P(c)}{P(d)}$

- Substitute to the classification objective:

$$\arg\max_{c \in C} P(c|d) = \arg\max_{c \in C} \frac{P(d|c)\,P(c)}{P(d)}$$

- Because arg max searches for specific class c that results in the largest overall value, and P(d) is constant for a given data point, dividing by a positive constant scales all values equally but does not change which value is the largest. Therefore, we can drop P(d) for optimization. We have final objective:

$$\arg\max_{c \in C} P(d|c)\,P(c)$$

b)

In practice, the likelihood P(d|c) is calculated by multiplying the probabilities of many individual features. Because these individual probabilities are fractions between 0 and 1, multiplying dozens or hundreds of them together result in an infinitesimally small number. Computers lack the precision to store numbers this close to zero, leading to arithmetic underflow where the value rounds down to 0. By taking the logarithm, we convert the product of fractions into a sum of negative numbers ($\log(x \cdot y) = \log x + \log y$). Adding numbers is stable and prevents underflow.

The predicted class doesn't change because the logarithmic function is strictly monotonically increasing that if A is greater than B, than log(A) is also greater then log(B). Because the log transformation preserves the exact ordering of the values, the class c that maximizes the original probability product will be the exact same class that maximizes the sum of log probabilities.

2)

a) - The bag-of-word Assumption states that the position and order of word in a document do not mater, only their presence and frequency. The classifier here treats the text as a bag containing "almost" and "enjoyabe". It loses the syntactic context that "almost" is directly modifying "enjoyable" to dampen or negate its meaning

- The Conditional Independence Assumption assumes that the probability of seeing a specific word is independent of seeing any other word in the document, given the class label (c)

$$P(\text{almost}, \text{enjoyable} | c) = P(\text{almost} | c) \, P(\text{enjoyable} | c)$$

- Because of these assumptions, the classifier evaluates "enjoyable" in a vacuum. It recognizes "enjoyable" as strong indicator of the "Positive" class and multiplies that high probability into the final calculation. It completely misses that "almost" reverses or severely weakens that positivity. Therefore, the classifier is highly likely to falsely predict a positive sentiment

b)

A strategy is to expand the feature space by using bigrams (or n-grams) in addition to single words.

Instead of tokenizing the text into single words (almost, enjoyable) a bigram model pairs adjacent words together to create new, distinct features. The feature representation would become "almost_enjoyable" for this phrase.

By treating "almost_enjoyable" as a single, distinct vocabulary term, the classifier can learn a specific probability for that exact sequence. The model now independently learns that P("enjoyable" | Positive) is high, but P("almost_enjoyable" | Negative) might be higher than its positive counterpart. This effectively bypasses the conditional independence assumption for that specific pair of words, allowing the model to capture the local context and the modifier's impact on the adjective it precedes